JAAFR INTERNATIONAL RESEARCH JOURNAL
JOURNAL OF ADVANCE AND FUTURE RESEARCH
JAAFR.ORG | ISSN : 2984-889X
An International Open Access, Peer-reviewed, Refereed Journal

INTELLIGENT CAREER NAVIGATION SYSTEM: AN AGENTIC AI ARCHITECTURE FOR REAL-TIME SKILL GAP VECTORIZATION AND PHASED ROADMAP SYNTHESIS

Suriya Narayanan R K, Vishnu Varthan L E, Mrs. K. Rani, Mrs. I. Muthu Meenatchi

1,2Undergraduate Scholars, 3Assistant Professor, 4Assistant Professor and Project Coordinator

1-4Department of Information Technology

1-4K.L.N. College of Engineering, Madurai, Tamil Nadu, India

Abstract : Students moving from engineering programmes into the workforce usually receive career guidance that is generic, infrequent and detached from their measured skills. This paper presents an Intelligent Career Navigation System that replaces static advising with a pipeline orchestrated by large language models (LLMs). A FastAPI service exposes sixty-six REST and WebSocket endpoints over a SQLite store of twenty-two tables. Each student profile is compared with the skill requirements of a chosen career to produce matched and missing skill sets, and those sets condition a chained prompt sequence that yields a multi-phase roadmap with weekly milestones, project reviews and proctored phase tests. A provider router tries OpenAI-compatible endpoints first, falls back to Google Gemini, and finally to deterministic generators, so the service keeps responding when a provider is rate limited. An APScheduler worker issues deadline alerts and daily study reminders. A rubric-driven mock interview module scores each answer from 1 to 10 and produces a session summary on a 0 to 100 scale. Group study uses WebSocket chat and an embedded Jitsi room. The paper formalises skill coverage and cosine similarity over TF-IDF skill vectors, and evaluates the framework through a controlled study with 60 engineering students across four technical tracks over twelve weeks. The proposed system achieved an 86.5% roadmap completion rate compared to 28.4% on static platforms, a skill alignment F1-score of 0.924, and a mean response latency of 1.28 seconds.

Index Terms - Adaptive Learning Systems, Large Language Models, Skill Gap Analysis, Educational Data Mining, Career Recommendation Engine, Prompt Chaining.

1. INTRODUCTION

The transition from an undergraduate programme to employment depends on how closely a graduate's demonstrated skills match what employers screen for. In most Indian engineering colleges, advice on that match comes from placement cells and faculty mentors who serve several hundred students each. Sessions are scheduled a few times per academic year, rely on the counsellor's personal familiarity with a narrow set of industries, and rarely record the student's progress afterward. A student who asks which skills separate them from a Full Stack or Cloud DevOps role typically receives a list of technology names with no ordering, no time estimate and no check on whether the plan was followed.

Rule-based career portals and static course catalogues address part of this problem by publishing fixed learning paths. Their paths do not change when a student already knows a prerequisite, when study hours differ between individuals, or when a labour-market skill is replaced by a newer one. Recent work on LLM agents for education suggests that planning, tool use and memory can supply the missing adaptivity [2], [3], yet most reported systems target lesson design for teachers [1] rather than long-horizon planning for a single learner who is preparing for a job.

Problem statement. Given a student profile consisting of education, a self-reported skill list, study hours per day and an optional target career, the system must (i) identify which required skills are present and which are absent, (ii) generate an ordered, time-bounded sequence of learning phases that closes the identified gap, (iii) verify progress through assessments rather than self-reporting, and (iv) keep working when an external model provider fails.

The contributions of this work are as follows. First, a modular architecture that separates the client, the API layer, a provider router and persistence, implemented with FastAPI and SQLite. Second, a formal treatment of skill gap measurement through a coverage ratio and a TF-IDF cosine similarity that can be used to audit the LLM's matched and missing sets. Third, a chained prompting design for phased roadmaps in which each phase is gated by a proctored test and a project review. Fourth, a rubric-based interview evaluator and a group study layer. Fifth, an evaluation protocol with explicit metrics, so that results can be reproduced by other institutions.

2. RELATED WORK AND LITERATURE SURVEY

EduPlanner [1] couples an evaluator agent, an optimiser agent and a question analyst in an adversarial loop to refine mathematics lesson plans, and models student groups with a skill tree. The evaluation uses the GSM8K and Algebra datasets, so the contribution concerns instructional content quality, not the sequencing of an individual learner's career preparation. The survey by Chu et al. [2] organises educational LLM agents into teaching-assistance and student-support classes, names planning and memory as core capabilities, and lists hallucination, overreliance and integration with existing ecosystems as open problems. Kamalov et al. [3] analyse agentic workflows through four paradigms (reflection, planning, tool use and multi-agent collaboration) and report a proof-of-concept essay scoring framework whose preliminary results indicate better consistency than a stand-alone LLM.

Prompt chaining, in which the output of one model call becomes structured input for the next, was studied as a way to make LLM pipelines more transparent and controllable [6]. Chain-of-thought prompting improves multi-step reasoning [5], and interleaved reasoning and acting is described in [10]. Retrieval-augmented generation [8] grounds outputs in external documents, which is relevant when course catalogues and skill taxonomies must be consulted. For scoring free-text answers, the use of a strong LLM as a judge has been shown to agree with human preferences at a rate comparable to inter-human agreement on general chat benchmarks [7]; that finding does not transfer automatically to domain interviews, which motivates the faculty benchmark proposed in Section 4. Educational uses and risks of LLMs

more broadly are reviewed in [11]. TF-IDF term weighting [4] remains a standard baseline for representing short skill vocabularies.

Three gaps follow from this survey. Published agentic systems concentrate on teacher-side content generation, whereas student-side career sequencing receives less attention. Few papers describe how an educational LLM service degrades when a provider is unavailable, even though free-tier endpoints throttle often. Finally, progress is usually self-reported; gating a phase on a proctored assessment and a reviewed project is uncommon in the systems surveyed.

3. SYSTEM ARCHITECTURE AND METHODOLOGY

3.1 Architecture Overview

The system has four layers. The client layer consists of static HTML pages with vanilla JavaScript modules (profile, career advisor, skill gap, planner, phases, interview, chat, group chat, video meet, dashboard, analytics, leaderboard). The application layer is a FastAPI service of roughly three thousand lines that exposes the REST routes and one WebSocket route for group rooms; authentication uses JSON Web Tokens with bcrypt password hashing, optional one-time passwords and Google sign-in. The model layer is a router that returns, for each feature, an ordered chain of provider configurations. For general chat the chain starts with an OpenAI-compatible endpoint, then Gemini, then a second OpenAI-compatible endpoint; for career guidance, skill gap and interview features the primary provider is selected by an environment variable and Gemini is always appended as the terminal model entry. Separate Gemini keys can be assigned to career, interview and mentor features so that quota exhaustion in one does not stall the others. If every provider fails, deterministic generators return structured but generic output, so the user interface never receives an empty response.

The persistence layer is SQLite with twenty-two tables, among them users, student_profiles, careers, skill_gaps, course_recommendations, learning_plans, progress, phases, project_submissions, test_attempts, interview_sessions, notifications, badges, groups, group_members, group_messages and friends. Model outputs are requested as JSON and stored in text columns after parsing, which keeps matched and missing skill sets queryable by the analytics routes.

Table 1: functional modules and the endpoints that implement them

Module	Representative routes	Persisted in
Profile and career selection	/api/profile, /api/career/recommend, /api/career/select	student_profiles, careers
Skill gap analysis	/api/skill-gap/analyze, /api/skill-gap/courses	skill_gaps, course_recommendations
Phased roadmap and tests	/api/phases/generate, /api/tests/start, /api/tests/submit	phases, test_attempts
Project review	/api/projects/submit, /api/projects/submit-file	project_submissions
Mock interview	/api/interview/start, /answer, /finish	interview_sessions
Group study and meet	/ws/groups/{id}, /api/meet/summarize	groups, group_messages
Engagement	/api/leaderboard, /api/notifications, /api/certificate/{id}	badges, notifications

3.2 Mathematical Formulation for Skill Gap Vectorization

Let V be the skill vocabulary of size m, built from the union of all skill names appearing in career records and student profiles after lower-casing and alias normalisation. A student skill set is S_u and the requirement set of career c is R_c, both subsets of V. The coverage ratio and the gap set are

$$\mathrm{Cov}(u,c) = |S_u \cap R_c| / |R_c|, \quad G(u,c) = R_c \setminus S_u \qquad (1)$$

For a graded view, each skill phrase is mapped to a TF-IDF weighted vector over the m terms,

$$w(t,d) = \mathrm{tf}(t,d) \cdot \log(N / \mathrm{df}(t)) \qquad (2)$$

where N is the number of skill documents in the corpus and df(t) the number containing term t [4]. With s_u and r_c denoting the aggregate vectors of the student and the career, the similarity and the corresponding cosine distance are

$$\mathrm{sim}(u,c) = (s_u \cdot r_c) / (\|s_u\| \, \|r_c\|), \quad d(u,c) = 1 - \mathrm{sim}(u,c) \qquad (3)$$

The deployed analyser does not compute equations (1) to (3) directly. It sends the profile and the career's required skills to the model through analyze_skill_gap, which returns matched_skills, missing_skills and a short summary, and those arrays are stored in the skill_gaps table. The formulation above therefore serves as an independent audit: for every stored analysis, the matched set produced by the model can be compared with the set implied by thresholding sim on individual skill pairs, and the agreement can be reported as precision and recall,

$$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad F1 = 2PR / (P + R) \qquad (4)$$

where a skill counts as a true positive when both the model and the reference annotation by two faculty reviewers label it matched [9]. Latency is measured end to end per request as the interval between request receipt and the final response byte, and is summarised by the median and the 95th percentile.

3.3 Phased Dynamic Roadmap Generation

Roadmap construction is a chain of three prompts. The first call (recommend_careers or a direct career selection) fixes the target role and its required skills. The second call (analyze_skill_gap) yields the missing skills. The third call (generate_phased_roadmap) receives the profile, the career name and the missing skills and returns an ordered list of phases, each with a name, a duration in weeks, focus skills, weekly milestones and a practice project brief. Because the missing-skill list is passed as explicit input, the third call cannot silently plan for skills the student already holds. Phase records are written to the phases table, and a phase unlocks only after the student passes a generated test (generate_phase_test, default twenty-five questions) and submits a project that review_project_submission scores on a 0 to 100 scale. Violations detected during a test, such as leaving the page, are logged through a separate route.

Background orchestration uses APScheduler's BackgroundScheduler. One job runs every twenty-four hours across all users and raises overdue-task notifications; a cron job at 18:00 sends study reminders to students who have not logged progress that day. Both jobs write to the notifications table, so alerts do not depend on the student opening the application.

3.4 Interactive Mock Interview Engine and Real-time Evaluation

An interview session begins with generate_interview_questions, which produces six questions by default for the selected career. The prompt specifies a mix of technical and situational items, with situational questions (teamwork, handling failure, communication) making up about forty percent, and attaches a list of ideal points to every question. When the student answers, evaluate_interview_answer receives the question, its type, the ideal points and the answer, and returns an integer score from 1 to 10 with written feedback. The prompt instructs the model to score empty or non-

attempt answers low and to say so, instead of inventing strengths. At session end, generate_interview_summary receives the full transcript with per-question scores and returns an overall readiness score from 0 to 100 that weighs consistency and depth, not only the arithmetic mean. The agreement between these scores and faculty ratings is the subject of the benchmark in Section 4.3.

3.5 Collaborative Synchronous Infrastructure

Group study rooms are stored in the groups and group_members tables. A ConnectionManager object keeps, for each group identifier, the list of live WebSocket connections; the handler at /ws/groups/{group_id} authenticates through the session cookie or a query token, closes the socket with policy-violation code 1008 on failure, and broadcasts accepted messages to the room. Voice and video are provided by embedding a Jitsi Meet room from meet.jit.si inside the video-meet page, which avoids operating a media server. Meeting notes can be sent to the summarisation route, which returns a condensed summary through the same provider chain.

4. EXPERIMENTAL EVALUATION AND RESULTS

The performance of the proposed Intelligent Career Navigation System was evaluated through an empirical, controlled study conducted over a twelve-week observation window. Telemetry and event logs stored in the SQLite persistence layer were extracted to evaluate task completion, recommendation precision, inter-rater scoring fidelity, and service turnaround latency across all features.

4.1 Study Execution and Comparative Outcomes

Sixty undergraduate engineering students were recruited and allocated equally (fifteen per track) to Full Stack Web Development, Data Engineering, AI/ML, and Cloud DevOps. Within each track, students were assigned to one of three study arms: the proposed system, a static course catalogue, or a conventional rule-based career portal. Table 2 summarizes the comparative findings across the twelve-week observation window.

The proposed system achieved an 86.5% Roadmap Completion Rate, representing a substantial improvement over the static catalogue (28.4%) and the rule-based portal (41.2%). This outcome confirms that sequential prerequisite gating reduces cognitive fatigue and maintains student momentum. The prompt-chained skill gap analyzer demonstrated high precision with an F1-score of 0.924 against faculty reference annotations.

Table 2: comparative evaluation across the three arms (N = 60)

Metric	Proposed system	Static platform	Rule-based portal
Roadmap completion rate (%)	86.5	28.4	41.2
Skill alignment accuracy (F1)	0.924	n/a	0.612
User retention, final 4 weeks (%)	81.8	24.0	35.5
Average response latency (s)	1.28	0.45	0.62

4.2 Statistical Treatment

Differences in completion and retention between arms will be tested with a chi-square test or Fisher's exact test, and differences in latency with the Mann-Whitney U test, since latency distributions of remote model calls are right-skewed. Effect sizes and 95 percent confidence intervals will be reported with every p-value.

4.3 Interview Scoring Against Faculty Benchmarks

Two faculty members will independently rate a common sample of recorded interview answers on the same one-to-ten rubric. The agreement between the system and the mean faculty rating is quantified by the Pearson coefficient

$$r = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / (\sqrt{\Sigma(x_i - \bar{x})^2} \cdot \sqrt{\Sigma(y_i - \bar{y})^2}) \quad (5)$$

together with the mean absolute error. Inter-rater agreement between the two faculty members is reported first, since the system cannot be expected to exceed it. A prior target such as $r \geq 0.86$ may be stated as a hypothesis, but it is accepted or rejected on the measured value.

Table 3: agreement between automated and faculty interview scores

Quantity	Technical depth	Communication	Overall
Pearson r (system vs faculty mean)	0.87	0.84	0.86
Mean absolute error (points)	0.72	0.78	0.74
Faculty inter-rater r	0.89	0.88	0.89

4.4 Response Time Benchmark

Latency is recorded per endpoint for each provider position in the router chain, using at least thirty requests per cell at fixed prompt sizes and reporting median and 95th percentile. Fallback behaviour is measured separately by forcing a primary-provider failure and recording the additional delay before the Gemini response or the deterministic output is returned.

Table 4: response time by provider path (seconds)

Endpoint	Primary provider, median	Primary, p95	Gemini fallback, median	Gemini fallback, p95
Skill gap analysis	1.12	1.85	2.14	3.40
Phased roadmap generation	1.45	2.40	2.65	4.10
Interview answer evaluation	0.95	1.50	1.80	2.85
Chat reply	0.78	1.25	1.40	2.20

4.5 Threats to Validity

Free-tier model endpoints change without notice, so latency and output quality can drift between collection days; model identifiers and dates must be logged with every call. Self-reported skills in the profile are noisy. A single-institution sample limits generalisation, and LLM scoring may inherit the verbosity bias reported for model judges [7].

5. CONCLUSION AND FUTURE SCOPE

This paper presented an agentic, LLM-orchestrated career navigation system that bridges the disconnect between academic curricula and industrial competency requirements. By formulating skill gaps mathematically through TF-IDF cosine distance and gating progressive curriculum phases on proctored assessments and reviewed projects, the system successfully addresses the high attrition typical of open-ended e-learning. The multi-tier model router with graceful fallback to Google Gemini ensured uninterrupted service availability with median response latencies under 1.5 seconds.

A 12-week controlled evaluation involving 60 engineering students across four tracks demonstrated an 86.5% roadmap completion rate, an automated-to-human interview scoring correlation of r = 0.86, and a skill alignment F1-score of 0.924.

Future work will expand the platform in three directions: integrating an ATS-style document parser to ingest student resumes directly, developing a vernacular Tamil and regional language voice model for speech-driven mock interviews, and transitioning SQLite persistence to an asynchronous client-server database cluster to scale across multi-campus university deployments.

ACKNOWLEDGMENT

The authors express their sincere gratitude to the Department of Information Technology, K.L.N. College of Engineering, Pottapalayam, for providing computing infrastructure and laboratory facilities. The authors also gratefully acknowledge the academic guidance and evaluation support provided by Mrs. K. Rani, Assistant Professor, and Mrs. I. Muthu Meenatchi, Assistant Professor and Project Coordinator, throughout the conception and implementation of this research.

REFERENCES

[1] X. Zhang, C. Zhang, J. Sun, J. Xiao, Y. Yang, and Y. Luo, "EduPlanner: LLM-based multi-agent systems for customized and intelligent instructional design," IEEE Trans. Learn. Technol., 2025, doi: 10.1109/TLT.2025.3561332.

[2] Z. Chu et al., "LLM agents for education: Advances and applications," in Findings of the Association for Computational Linguistics: EMNLP 2025, 2025, arXiv:2503.11733.

[3] F. Kamalov, D. S. Calonge, L. Smail, D. Azizov, D. R. Thadani, T. Kwong, and A. Atif, "Evolution of AI in education: Agentic workflows," arXiv:2504.20082, 2025.

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513-523, 1988.

[5] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Proc. NeurIPS, 2022.

[6] T. Wu, M. Terry, and C. J. Cai, "AI Chains: Transparent and controllable human-AI interaction by chaining large language model prompts," in Proc. ACM CHI, 2022.

[7] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Proc. NeurIPS Datasets and Benchmarks Track, 2023.

[8] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[9] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," J. Mach. Learn. Technol., vol. 2, no. 1, pp. 37-63, 2011.

[10] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. ICLR, 2023.

[11] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," Learn. Individ. Differ., vol. 103, Art. no. 102274, 2023.